

MBGA API Documentation — Full Endpoint Reference

Consolidated, per-endpoint reference for every API required by the **Customer app** and the **Merchant app**. This file restates **API_SPEC.md** in a uniform, one-endpoint-per-section format (method, auth, payload, response, errors) so each API can be read in isolation. **API_SPEC.md** remains the source of truth for data model definitions, enums, and business rules — this file is the quick-reference companion to it.

Base URL: `env.apiUrl` (e.g. `https://api.mbga.in/api/v1`) — all paths below are relative to it. **Transport:** HTTPS, JSON (`Content-Type: application/json; charset=utf-8`); multipart only for §17.1. **Auth:** `Authorization: Bearer <token>` header on every endpoint marked **Auth: Yes**. **Money:** integer whole rupees, GST-inclusive. **Dates:** ISO-8601 UTC. **IDs:** opaque strings.

Legend

Column	Meaning
Used by	Customer = Customer app only · Merchant = Merchant app only · Both = shared by both
Auth	No = public · Token = any signed-in user · Permission = requires the named permission/role

Master index (46 endpoints)

#	Method	Path	Used by	Auth
1	POST	<code>/auth/otp/send</code>	Both	No
2	POST	<code>/auth/otp/verify</code>	Both	No
3	POST	<code>/auth/otp/resend</code>	Both	No
4	GET	<code>/auth/me</code>	Both	Token
5	POST	<code>/auth/logout</code>	Both	Token
6	POST	<code>/customers/register</code>	Customer	Token (customer, NEW)
7	GET	<code>/customers/me</code>	Customer	Token (customer)

#	Method	Path	Used by	Auth
8	GET	/dashboard/customer	Customer	Token (customer, APPROVED)
9	GET	/pricing/customers/{customerId}	Both	Token (customer own / pricing.view)
10	GET	/orders/statuses	Both	Token
11	GET	/orders/cutoff	Both	Token
12	POST	/orders/quote	Both	Token (customer own / orders.create)
13	POST	/orders	Both	Token (customer own / orders.create)
14	GET	/orders	Both	Token (customer own / orders.view)
15	GET	/orders/{id}	Both	Token (owner / orders.view)
16	POST	/customers	Merchant	customers.create
17	GET	/customers	Merchant	customers.view
18	GET	/customers/{id}	Merchant	customers.view
19	GET	/customers/{id}/eligibility	Merchant	orders.create
20	GET	/kyc/applications	Merchant	customers.kyc.review
21	GET	/kyc/applications/{id}	Merchant	customers.kyc.review
22	POST	/kyc/applications/{id}/decision	Merchant	customers.kyc.review
23	GET	/pricing/months	Merchant	pricing.view
24	PUT	/pricing/months/{id}/entries	Merchant	pricing.manage
25	GET	/deliveries	Merchant	delivery.view
26	GET	/deliveries/{id}	Merchant	delivery.view
27	POST	/deliveries/{id}/dispatch	Merchant	delivery.confirm
28	POST	/deliveries/{id}/confirm	Merchant	delivery.confirm
29	GET	/inventory	Merchant	inventory.view
30	GET	/inventory/movements	Merchant	inventory.view
31	POST	/inventory/movements	Merchant	inventory.adjust
32	GET	/customers/{id}/payments/summary	Both	Token (customer own / payments.view)

#	Method	Path	Used by	Auth
33	GET	/invoices	Both	Token (customer own / payments.view)
34	GET	/payments	Both	Token (customer own / payments.view)
35	GET	/payments/{id}	Both	Token
36	POST	/payments	Merchant	payments.record
37	GET	/expenses	Merchant	expenses.view
38	POST	/expenses	Merchant	expenses.add
39	GET	/reports/{type}	Merchant	reports.view
40	GET	/notifications	Both	Token
41	GET	/notifications/{id}	Both	Token
42	POST	/notifications/{id}/read	Both	Token
43	POST	/notifications/read-all	Both	Token
44	GET	/dashboard/merchant	Merchant	dashboard.view
45	POST	/documents	Both	Token (customer NEW/PENDING or customers.create)
46	GET	/documents/{fileId}/url	Merchant	Token (owner or customers.kyc.review)

Table of contents

1. [Authentication \(1–5\)](#)
2. [Customer self-service \(6–9\)](#)
3. [Orders \(10–15\)](#)
4. [Customer management — merchant \(16–19\)](#)
5. [KYC review — merchant \(20–22\)](#)
6. [Pricing management — merchant \(23–24\)](#)
7. [Delivery & dispatch — merchant \(25–28\)](#)
8. [Warehouse inventory — merchant \(29–31\)](#)
9. [Payments & invoices \(32–36\)](#)
10. [Expenses — merchant \(37–38\)](#)
11. [Reports — merchant \(39\)](#)
12. [Notifications \(40–43\)](#)
13. [Merchant dashboard \(44\)](#)

- 14. [Document upload \(45–46\)](#)
- 15. [Common error envelope](#)

1. Authentication

1. `POST /auth/otp/send`

Used by: Both · **Auth:** No · **Frontend:** `authService.sendOtp`

Request payload

Field	Type	Required	Notes
mobile	string	✓	Exactly 10 digits
appTarget	"customer" "merchant"	✓	Which app is calling

```
{ "mobile": "9000000001", "appTarget": "customer" }
```

Response `200`

Field	Type	Notes
requestId	string	Echoed back on verify/resend
expiresInSeconds	integer	120
resendAfterSeconds	integer	30
devOtp	string	Mock only — must not exist in production

```
{ "requestId": "otp-m1x9k2", "expiresInSeconds": 120, "resendAfterSeconds": 30 }
```

Errors: `400` invalid mobile · `403 WRONG_APP` (staff number on customer app or vice-versa) · `403 USER_INACTIVE` · `429` too many requests · `503` SMS provider down.

2. `POST /auth/otp/verify`

Used by: Both · **Auth:** No · **Frontend:** `authService.verifyOtp`

Request payload

Field	Type	Required
requestId	string	✓
mobile	string	✓
otp	string (4 digits)	✓
appTarget	"customer" "merchant"	✓

Response 200 — AuthSession

Field	Type	Required
token	string	✓
expiresAt	ISO date	✓
user	AuthUser	✓

AuthUser: id, mobile, name, role

(CUSTOMER|MANAGER|SALESPERSON|GODOWN_INCHARGE|ACCOUNTANT), permissions [], status (ACTIVE|INACTIVE), appTarget, customerId?, customerStatus? (NEW|PENDING|APPROVED|REJECTED|SUSPENDED), branchName? (staff only).

```
{
  "token": "eyJhbGciOi...",
  "expiresAt": "2026-06-22T09:12:00.000Z",
  "user": {
    "id": "USR-C001", "mobile": "9000000001", "name": "Rajendra Sharma",
    "role": "CUSTOMER", "permissions": [], "status": "ACTIVE", "appTarget":
    "customer",
    "customerId": "CUST-001", "customerStatus": "APPROVED"
  }
}
```

Unknown number on the Customer app → create user with **customerStatus: "NEW"**, no **customerId** (app routes to registration).

Errors: 400 OTP_INVALID ("Incorrect OTP. 2 attempts left.") · 400 OTP_EXPIRED · 404 unknown request · 403 WRONG_APP / 403 USER_INACTIVE.

3. POST /auth/otp/resend

Used by: Both · **Auth:** No · **Frontend:** authService.resendOtp

Request: { "requestId": string, "appTarget": "customer" | "merchant" }

Response 200: same shape as endpoint 1 (new requestId). **Errors:** 404 unknown request · 429 before resendAfterSeconds elapses.

4. GET /auth/me

Used by: Both · **Auth:** Token · **Frontend:** `authService.restoreSession` (app launch)

Request: none. **Response 200:** `AuthUser` (fresh `customerStatus/permissions`). **Errors:** 401 → app signs the user out.

5. POST /auth/logout

Used by: Both · **Auth:** Token · **Frontend:** `authService.logout`

Request: empty body. **Response:** 204 (no content). Backend must invalidate the token.

2. Customer self-service

6. POST /customers/register

Used by: Customer · **Auth:** Token (customer, `customerStatus: NEW`) · **Frontend:** `customerService.register`

Request payload — CustomerRegistrationRequest

Field	Type	Required	Notes
customerType	"RETAIL" "INDUSTRIAL"	✓	
businessName	string	✓	
ownerName	string	✓	
mobile	string	✓	Must equal token's mobile
email	string		Valid email
deliveryAddress	Address	✓	{ line1, line2?, city, state, pincode }
documents	{ type, number, fileName }[]	✓	RETAIL: AADHAAR (12 digits) + PAN (ABCDE1234F). INDUSTRIAL: FSSAI (14 digits) + GST (15-char GSTIN)

Field	Type	Required	Notes
sites	{ name, address, contactName?, contactMobile? }[]		INDUSTRIAL only

```
{
  "customerType": "RETAIL",
  "businessName": "Test Bakery", "ownerName": "Ravi Kumar", "mobile": "9000000099",
  "email": "ravi@testbakery.in",
  "deliveryAddress": { "line1": "12 MG Road", "line2": "", "city": "Indore", "state": "Madhya Pradesh", "pincode": "452001" },
  "documents": [
    { "type": "AADHAAR", "number": "123412341234", "fileName": "aadhaar_m1x9k2.pdf" },
    { "type": "PAN", "number": "ABCDE1234F", "fileName": "pan_m1x9k3.pdf" }
  ]
}
```

Response 201 — Customer with `accountStatus: "PENDING"`, `kycStatus: "PENDING"`, documents `UPLOADED` with masked numbers.

```
{
  "id": "CUST-011", "code": "MBGA-R-0011", "customerType": "RETAIL",
  "businessName": "Test Bakery", "ownerName": "Ravi Kumar", "mobile": "9000000099",
  "email": "ravi@testbakery.in",
  "accountStatus": "PENDING", "kycStatus": "PENDING", "pricingTier": "STANDARD",
  "deliveryAddress": { "line1": "12 MG Road", "city": "Indore", "state": "Madhya Pradesh", "pincode": "452001" },
  "sites": [],
  "documents": [
    { "type": "AADHAAR", "numberMasked": "XXXXXXXX1234", "fileName": "aadhaar_m1x9k2.pdf", "uploadedAt": "2026-06-15T09:12:00.000Z", "status": "UPLOADED" },
    { "type": "PAN", "numberMasked": "XXXXXX234F", "fileName": "pan_m1x9k3.pdf", "uploadedAt": "2026-06-15T09:12:00.000Z", "status": "UPLOADED" }
  ],
  "registeredAt": "2026-06-15T09:12:00.000Z", "runningBalance": 0
}
```

Side effects: creates a `PENDING KycApplication`, sets `customerStatus = PENDING`, notifies customer ("Application received") and merchant bucket ("New KYC application").

Errors: 422 `fieldErrors` (`businessName`, `ownerName`, `email`, `address.line1/city/pincode/state`, `doc.AADHAAR.number`, `doc.PAN.file`, ...) · 409 mobile already registered · 401.

7. GET /customers/me

Used by: Customer · **Auth:** Token (customer) · **Frontend:** `customerService.getMyProfile`

Request: none. **Response 200:** `Customer` (full shape, see §2 of `API_SPEC.md` §3.6). **Errors:** `404` if not registered.

8. GET /dashboard/customer

Used by: Customer · **Auth:** Token (customer, APPROVED) · **Frontend:** `dashboardService.getCustomerDashboard`

Response 200 — `CustomerDashboard`

Field	Type	Notes
customer	<code>Customer</code>	
currentOrder	<code>Order</code> absent	Newest non-terminal order
recentOrder	<code>Order</code> absent	Newest terminal order
pricing	<code>CustomerPrice[]</code>	
paymentSummary	<code>CustomerPaymentSummary</code>	{ <code>totalInvoiced</code> , <code>totalPaid</code> , <code>runningBalance</code> , <code>overdueAmount</code> , <code>lastPaymentAt?</code> }
unreadNotifications	integer	

```
{
  "customer": { "...": "Customer" },
  "currentOrder": { "id": "ORD-2606-0148", "status": "OUT_FOR_DELIVERY", "...":
"Order" },
  "recentOrder": { "id": "ORD-2606-0141", "status": "DELIVERED", "...": "Order" },
  "pricing": [ { "cylinderType": "LPG_19KG", "cylinderLabel": "19 KG", "price": 1800,
"tier": "STANDARD", "gstPercent": 18, "effectiveFrom": "2026-06-01T00:00:00.000Z",
"effectiveTo": "2026-06-30T23:59:59.000Z" } ],
  "paymentSummary": { "totalInvoiced": 14940, "totalPaid": 11400, "runningBalance":
3540, "overdueAmount": 0, "lastPaymentAt": "2026-06-11T14:25:00.000Z" },
  "unreadNotifications": 2
}
```

9. GET /pricing/customers/{customerId}

Used by: Both (customer views own; merchant views any for `pricing.view`) · **Auth:** Token · **Frontend:** `pricingService.getCustomerPricing`

Path param: `customerId`. **Response 200** — `CustomerPrice[]` from the ACTIVE pricing month, filtered to cylinders the customer type may order.

```
[
  { "cylinderType": "LPG_5KG", "cylinderLabel": "5 KG", "price": 490, "tier":
    "STANDARD", "gstPercent": 18, "effectiveFrom": "2026-06-01T00:00:00.000Z",
    "effectiveTo": "2026-06-30T23:59:59.000Z" },
  { "cylinderType": "LPG_19KG", "cylinderLabel": "19 KG", "price": 1800, "tier":
    "STANDARD", "gstPercent": 18, "effectiveFrom": "...", "effectiveTo": "..." }
]
```

Errors: 404 customer / no active pricing.

3. Orders

10. GET /orders/statuses

Used by: Both · **Auth:** Token · **Frontend:** `orderService.getOrderStatuses`

Response 200 — `OrderStatusMeta[]` sorted by `sequence` (drives the tracking timeline; no transitions hard-coded client-side).

```
[
  { "code": "PLACED", "label": "Order Placed", "sequence": 1, "tone": "info",
    "description": "We have received your order.", "isTerminal": false },
  { "code": "CONFIRMED", "label": "Confirmed", "sequence": 2, "tone": "info",
    "description": "Order confirmed and scheduled for delivery.", "isTerminal": false },
  { "code": "PREPARING", "label": "Preparing / Dispatch", "sequence": 3, "tone":
    "pending", "description": "Cylinders are being allocated at the godown.",
    "isTerminal": false },
  { "code": "OUT_FOR_DELIVERY", "label": "Out for Delivery", "sequence": 4, "tone":
    "warning", "description": "Your cylinders are on the way.", "isTerminal": false },
  { "code": "DELIVERED", "label": "Delivered", "sequence": 5, "tone": "success",
    "description": "Delivered and confirmed.", "isTerminal": true },
  { "code": "CANCELLED", "label": "Cancelled", "sequence": 99, "tone": "error",
    "description": "This order was cancelled.", "isTerminal": true }
]
```

11. GET /orders/cutoff

Used by: Both · **Auth:** Token · **Frontend:** `orderService.getCutoffInfo`

Response 200 — `CutoffInfo`. Rule: order before **16:00 IST** → next-day delivery; after → day after tomorrow.

```
{ "cutoffTime": "16:00", "withinCutoff": true, "scheduledDeliveryDate": "2026-06-
16T09:00:00.000Z", "message": "Order before 4:00 PM for next-day delivery." }
```

12. POST /orders/quote

Used by: Both · **Auth:** Token (customer own id, or staff `orders.create`) · **Frontend:** `orderService.getQuote`

Request payload — `OrderQuoteRequest`

Field	Type	Required	Notes
customerId	string	✓	
items	{ <code>cylinderType</code> , <code>quantity</code> }[]	✓	≥1 line, duplicates merged, quantity limits per cylinder (§3, §14 of API_SPEC.md), Hippo industrial-only

```
{ "customerId": "CUST-001", "items": [ { "cylinderType": "LPG_19KG", "quantity": 2 }, { "cylinderType": "LPG_5KG", "quantity": 1 } ] }
```

Response 200 — `OrderQuote`

Field	Type
items	<code>OrderItem</code> []
totalCylinders	integer
subtotal	Money
gstPercent	number
gstAmount	Money
totalAmount	Money
cutoff	<code>CutoffInfo</code>

```
{
  "items": [
    { "cylinderType": "LPG_5KG", "cylinderLabel": "5 KG", "quantity": 1,
      "unitPrice": 490, "lineTotal": 490 },
    { "cylinderType": "LPG_19KG", "cylinderLabel": "19 KG", "quantity": 2,
      "unitPrice": 1800, "lineTotal": 3600 }
  ],
  "totalCylinders": 3, "totalAmount": 4090, "gstPercent": 18, "subtotal": 3466,
  "gstAmount": 624,
  "cutoff": { "cutoffTime": "16:00", "withinCutoff": true, "scheduledDeliveryDate":
    "2026-06-16T09:00:00.000Z", "message": "Order before 4:00 PM for next-day delivery." }
}
```

Errors: 422 `fieldErrors.items` "Add at least one cylinder" · 422 `fieldErrors.<CYLINDER_TYPE>` "Enter 1–50" · 422 Hippo restricted to industrial · 404 customer.

13. POST /orders

Used by: Both · **Auth:** Token (customer own id + APPROVED, or staff `orders.create`) · **Frontend:** `orderService.createOrder`

Request payload — `CreateOrderRequest`

Field	Type	Required	Notes
<code>customerId</code>	<code>string</code>	✓	
<code>items</code>	<code>{ cylinderType, quantity }[]</code>	✓	Same rules as quote
<code>orderMode</code>	<code>"NEW" "REPEAT"</code>	✓	
<code>deliverySiteId</code>	<code>string</code>	conditional	Required for INDUSTRIAL; must be one of <code>customer.sites[].id</code>
<code>source</code>	<code>"CUSTOMER_APP" "MERCHANT_APP"</code>	✓	Backend may also infer from role

```
{ "customerId": "CUST-002", "items": [ { "cylinderType": "LPG_47_5KG_L", "quantity": 6 }, { "cylinderType": "LPG_47_5KG_V", "quantity": 2 } ], "orderMode": "NEW", "deliverySiteId": "SITE-002A", "source": "MERCHANT_APP" }
```

Response 201 — `Order` (`status: "PLACED"`, `statusHistory[0]` populated, `deliverySlot` assigned).

```
{  "id": "ORD-2606-0151", "orderNumber": "ORD-2606-0151",  "customerId": "CUST-002", "customerName": "Apex Foods Pvt Ltd", "customerType": "INDUSTRIAL",  "items": [    { "cylinderType": "LPG_47_5KG_L", "cylinderLabel": "47.5 KG L", "quantity": 6, "unitPrice": 4375, "lineTotal": 26250 },    { "cylinderType": "LPG_47_5KG_V", "cylinderLabel": "47.5 KG V", "quantity": 2, "unitPrice": 4410, "lineTotal": 8820 }  ],  "totalCylinders": 8, "itemsSummary": "6 × 47.5 L · 2 × 47.5 V",  "orderMode": "NEW", "source": "MERCHANT_APP",  "deliverySiteId": "SITE-002A", "deliverySiteName": "Plant A – Sanwer Road",  "deliveryAddress": { "line1": "Plot 22, Sector C, Sanwer Road Industrial Area", "city": "Indore", "state": "Madhya Pradesh", "pincode": "452015" },  "subtotal": 29720, "gstPercent": 18, "gstAmount": 5350, "totalAmount": 35070,  "status": "PLACED",  "statusHistory": [ { "status": "PLACED", "at": "2026-06-15T09:12:00.000Z", "by": "Priya Nair" } ],
```

```
"placedAt": "2026-06-15T09:12:00.000Z",
"cutoff": { "cutoffTime": "16:00", "withinCutoff": true, "scheduledDeliveryDate":
"2026-06-16T09:00:00.000Z", "message": "Order before 4:00 PM for next-day delivery."
},
"deliverySlot": "09:00 AM – 01:00 PM", "createdBy": "Priya Nair"
}
```

Side effects: notify customer ("Order placed") and merchant ("New order received"); update `customer.lastOrderAt`.

Errors: **403** "Only approved customers can place orders." · **422** `fieldErrors.deliverySiteId` · quote-validation errors (endpoint 12) · **404**.

14. GET /orders

Used by: Both (customers see only their own; staff need `orders.view`) · **Auth:** Token · **Frontend:** `orderService.listOrders`

Query params

Param	Type	Notes
customerId	string	Staff filter (ignored/forced for customer callers)
status	<code>OrderStatusCode</code> "ALL" "ACTIVE"	ACTIVE = all non-terminal statuses
search	string	Matches orderNumber, customerName, itemsSummary

Response **200** — `Order[]` newest first.

15. GET /orders/{id}

Used by: Both (owner or staff) · **Auth:** Token · **Frontend:** `orderService.getOrder`

Response **200** — `Order`. **Errors:** **404**.

4. Customer management — merchant

16. POST /customers

Used by: Merchant · **Auth:** `customers.create` (Manager, Salesperson) · **Frontend:** `customerService.createByStaff`

Request payload: same `CustomerRegistrationRequest` shape as endpoint 6, but `mobile` is the customer's number typed by staff.

Response 201 — `Customer` (`accountStatus: "PENDING"`), same shape as endpoint 6's response.

Backend rules: `mobile` belongs to staff → `422 fieldErrors.mobile`. Existing customer with that mobile → `409`. Creates the customer login (if needed) + a PENDING KYC application; notifies merchant bucket.

Errors: `422 (fieldErrors.mobile "This number belongs to MBGA staff")` · `409 "A customer with this mobile already exists."`

17. GET /customers

Used by: Merchant · **Auth:** `customers.view` · **Frontend:** `customerService.listCustomers`

Query params

Param	Type	Notes
search	string	businessName, ownerName, mobile, code
customerType	"RETAIL" "INDUSTRIAL" "ALL"	
accountStatus	CustomerAccountStatus "ALL"	

Response 200 — `Customer[]` sorted by `businessName` A→Z. Full `Customer` fields: `id`, `code`, `customerType`, `businessName`, `ownerName`, `mobile`, `email?`, `accountStatus`, `kycStatus`, `pricingTier`, `deliveryAddress`, `sites[]`, `documents[]`, `gstin?`, `registeredAt`, `approvedAt?`, `rejectionReason?`, `runningBalance`, `lastOrderAt?`.

18. GET /customers/{id}

Used by: Merchant · **Auth:** `customers.view` · **Frontend:** `customerService.getCustomer`

19. GET /customers/{id}/eligibility

Used by: Merchant · **Auth:** `orders.create` · **Frontend:**
`customerService.checkEligibility`

Response **200**

```
{ "eligible": false, "reasons": [ "Account is pending – only approved customers can order.", "KYC is not verified." ] }
```

Eligible ⇔ `accountStatus = APPROVED` and `kycStatus = VERIFIED`.

5. KYC review — merchant

20. GET /kyc/applications

Used by: Merchant · **Auth:** `customers.kyc.review` · **Frontend:**
`customerService.listKycApplications`

Query: `status = PENDING` (default) | `APPROVED` | `REJECTED` | `ALL`. **Response 200**
— `KycApplication[]` newest `submittedAt` first.

```
[
  {
    "id": "KYC-003", "customerId": "CUST-003", "customerType": "RETAIL",
    "businessName": "Royal Dhaba", "ownerName": "Gurpreet Singh", "mobile": "9000000003",
    "documents": [
      {
        "type": "AADHAAR", "numberMasked": "XXXX XXXX 9034", "fileName":
        "aadhaar.jpg", "uploadedAt": "2026-06-13T14:20:00.000Z", "status": "UPLOADED" },
      {
        "type": "PAN", "numberMasked": "XXXXXX7788K", "fileName": "pan.jpg",
        "uploadedAt": "2026-06-13T14:20:00.000Z", "status": "UPLOADED" }
    ],
    "deliveryAddress": {
      "line1": "NH-3 Bypass, Opp. Petrol Pump, Rau", "city":
      "Indore", "state": "Madhya Pradesh", "pincode": "453331" },
    "sites": [], "status": "PENDING", "submittedAt": "2026-06-13T14:20:00.000Z" }
]
```

21. GET /kyc/applications/{id}

Used by: Merchant · **Auth:** `customers.kyc.review` · **Frontend:**
`customerService.getKycApplication`

Response 200 — `KycApplication`. Documents opened via endpoint 46.

22. POST `/kyc/applications/{id}/decision`

Used by: Merchant · **Auth:** `customers.kyc.review` · **Frontend:**
`customerService.decideKyc`

Request payload — `KycDecisionRequest`

Field	Type	Required	Notes
decision	"APPROVE" "REJECT"	✓	
rejectionReason	string	if REJECT	≥10 chars

```
{ "decision": "REJECT", "rejectionReason": "GST certificate does not match the  
business name" }
```

Response 200 — updated `KycApplication`.

```
{ "id": "KYC-004", "customerId": "CUST-004", "status": "REJECTED", "reviewedAt":  
"2026-06-15T10:05:00.000Z", "reviewedBy": "Rajesh Verma",  
  "rejectionReason": "GST certificate does not match the business name", "...": "rest  
of KycApplication" }
```

Backend rules: only `PENDING` applications may be decided. `APPROVE` → customer
`accountStatus=APPROVED`, `kycStatus=VERIFIED`, documents `VERIFIED`. `REJECT` →
`REJECTED` everywhere + stored reason. Notifies customer.

Errors: `422` `fieldErrors.rejectionReason` · `409` already reviewed · `404`.

6. Pricing management — merchant

23. GET `/pricing/months`

Used by: Merchant · **Auth:** `pricing.view` · **Frontend:**
`pricingService.getPricingMonths`

Response 200 — **PricingMonth[]** newest month first (15 entries per month: 5 cylinder types × 3 tiers).

```
[
  { "id": "PRC-2026-06", "month": "2026-06", "label": "June 2026", "effectiveFrom":
    "2026-06-01T00:00:00.000Z", "effectiveTo": "2026-06-30T23:59:59.000Z",
    "status": "ACTIVE", "gstPercent": 18,
    "entries": [
      { "cylinderType": "LPG_5KG", "tier": "STANDARD", "bpclBaseRate": 445,
        "tierMarkup": 45, "customerPrice": 490 }
    ],
    "updatedBy": "Rajesh Verma", "updatedAt": "2026-06-01T09:30:00.000Z" }
]
```

24. PUT **/pricing/months/{pricingMonthId}/entries**

Used by: Merchant · **Auth:** **pricing.manage** (Manager, Accountant) · **Frontend:** **pricingService.updatePriceEntry**

Request payload — **UpdatePriceEntryRequest**

Field	Type	Required	Notes
cylinderType	CylinderTypeCode	✓	
tier	PricingTier	✓	
bpclBaseRate	Money	✓	> 0
tierMarkup	Money	✓	≥ 0

customerPrice = **bpclBaseRate** + **tierMarkup** (GST-inclusive). Only **ACTIVE/DRAFT** months are editable.

```
{ "cylinderType": "LPG_19KG", "tier": "STANDARD", "bpclBaseRate": 1700, "tierMarkup":
  110 }
```

Response 200 — full updated **PricingMonth**. **Errors:** **422** "Base rate must be positive and markup cannot be negative." · **409** archived month · **404**.

7. Delivery & dispatch — merchant

25. GET **/deliveries**

Used by: Merchant · **Auth:** `delivery.view` · **Frontend:**

`deliveryService.listDeliveries`

Query: `status = SCHEDULED | DISPATCHED | DELIVERED | FAILED | ALL`, search (slip id, orderNumber, customerName, vehicleNumber). **Response 200** — `DeliverySlip[]` ordered DISPATCHED → SCHEDULED → FAILED → DELIVERED, then newest `scheduledDate`.

```
[
  {
    "id": "DS-0148", "orderId": "ORD-2606-0148", "orderNumber": "ORD-2606-0148",
    "customerId": "CUST-001", "customerName": "Sharma General Store",
    "deliveryAddress": { "line1": "Shop 14, Sapna Sangeeta Road", "city": "Indore",
      "state": "Madhya Pradesh", "pincode": "452001" },
    "deliverySiteName": "Registered delivery address",
    "items": [ { "cylinderType": "LPG_19KG", "cylinderLabel": "19 KG", "quantity": 4 } ],
    "itemsSummary": "4 × 19 KG · 6 × 5 KG", "cylindersAllocated": 10,
    "vehicleNumber": "MP09 GH 4521", "driverName": "Arjun Singh", "helperName":
      "Ramesh Kumar",
    "scheduledDate": "2026-06-15T09:00:00.000Z", "dispatchedAt": "2026-06-
      15T07:30:00.000Z",
    "emptiesCollected": 0, "pendingPickup": 4, "confirmationMethod": "OTP", "status":
      "DISPATCHED" }
]
```

26. GET /deliveries/{id}

Used by: Merchant · **Auth:** `delivery.view` · **Frontend:** `deliveryService.getDelivery`

Response 200 — `DeliverySlip`. **Errors:** 404.

27. POST /deliveries/{id}/dispatch

Used by: Merchant · **Auth:** `delivery.confirm` (Manager, Godown Incharge) · **Frontend:** `deliveryService.dispatch`

Request: empty body.

Backend rules (atomic): slip must be `SCHEDULED` (else 409); every line needs sufficient `filled` stock (else 409 with a "not enough stock" message and no counts change); writes one `DISPATCHED` stock movement per line; sets slip `DISPATCHED` + `dispatchedAt`; order → `OUT_FOR_DELIVERY`; notifies customer.

Response 200 — updated `DeliverySlip`. **Errors:** 409 insufficient stock / wrong state.

28. POST /deliveries/{id}/confirm

Used by: Merchant · **Auth:** `delivery.confirm` · **Frontend:** `deliveryService.confirmDelivery`

Request payload — `ConfirmDeliveryRequest`

Field	Type	Required	Notes
otp	string	if <code>confirmationMethod = OTP</code>	4 digits (mock accepts <code>4321</code>)
emptiesCollected	integer	✓	$0 \leq n \leq \text{cylindersAllocated}$
note	string		

```
{ "otp": "4321", "emptiesCollected": 3, "note": "Left 1 empty for next visit" }
```

Backend rules: slip must be `DISPATCHED`; validates OTP; sets `DELIVERED`, `deliveredAt`, `emptiesCollected`, `pendingPickup`; order → `DELIVERED`; writes `EMPTYES_COLLECTED` movement; notifies customer.

Response 200 — updated `DeliverySlip`. **Errors:** `422 fieldErrors.otp` "Incorrect OTP" · `422 fieldErrors.emptiesCollected` · `409` wrong state · `404`.

8. Warehouse inventory — merchant

29. GET /inventory

Used by: Merchant · **Auth:** `inventory.view` (Manager, Godown Incharge) · **Frontend:** `inventoryService.getInventory`

Response 200 — `InventorySnapshot` (`items[]`, `alerts[]`, `asOf`). Alerts derived server-side: `filled < reorderThreshold` → WARNING, `< threshold/2` → CRITICAL, `damaged ≥ 5` → INFO.

```
{
  "items": [
    { "cylinderType": "LPG_5KG", "cylinderLabel": "5 KG", "filled": 240, "empty": 118,
      "damaged": 4, "reorderThreshold": 150, "updatedAt": "2026-06-15T07:30:00.000Z" }
  ],
  "alerts": [
    { "id": "SA-LOW-LPG_47_5KG_L", "cylinderType": "LPG_47_5KG_L", "severity":
```

```
"WARNING", "message": "47.5 KG L filled stock (42) is below the reorder threshold (50)." }
],
"asOf": "2026-06-15T09:12:00.000Z"
}
```

30. GET /inventory/movements

Used by: Merchant · **Auth:** `inventory.view` · **Frontend:** `inventoryService.listMovements`

Query: `cylinderType` = `<code>` | ALL, `limit` (integer). **Response 200** — `StockMovement[]` newest first.

```
[
  { "id": "SM-0118", "type": "DISPATCHED", "cylinderType": "LPG_19KG",
    "cylinderLabel": "19 KG", "quantity": 4, "deltas": { "filled": -4 }, "referenceType":
    "DELIVERY", "referenceId": "DS-0148", "recordedBy": "Mohan Lal", "recordedAt": "2026-
    06-15T07:30:00.000Z" }
]
```

31. POST /inventory/movements

Used by: Merchant · **Auth:** `inventory.adjust` (Manager, Godown Incharge) · **Frontend:** `inventoryService.recordMovement`

Request payload — `RecordStockMovementRequest`

Field	Type	Required	Notes
type	"RECEIVED_FILLED" "SENT_TO_PLANT" "MARKED_DAMAGED" "CORRECTION"	✓	DISPATCHED/EMPTYES_COLLECTED rejected (system-generated)
cylinderType	CylinderTypeCode	✓	
quantity	integer ≥1	for first three types	
fromBucket	"filled" "empty" "damaged"	SENT_TO_PLANT / MARKED_DAMAGED	
bucket	StockBucket	CORRECTION	
newCount	integer ≥0	CORRECTION	

Field	Type	Required	Notes
note	string	CORRECTION required	
referenceId	string		Challan / vehicle no.

Effect on counts

type	deltas
RECEIVED_FILLED	filled +qty
SENT_TO_PLANT	fromBucket -qty
MARKED_DAMAGED	fromBucket -qty, damaged +qty
CORRECTION	bucket += (newCount - current)

```
{ "type": "RECEIVED_FILLED", "cylinderType": "LPG_19KG", "quantity": 60,
  "referenceId": "BPCL/CH/22871", "note": "Morning refill truck" }
```

Response 201 — created `StockMovement`.

```
{ "id": "SM-0119", "type": "RECEIVED_FILLED", "cylinderType": "LPG_19KG",
  "cylinderLabel": "19 KG", "quantity": 60, "deltas": { "filled": 60 },
  "note": "Morning refill truck", "referenceType": "CHALLAN", "referenceId":
  "BPCL/CH/22871", "recordedBy": "Mohan Lal", "recordedAt": "2026-06-15T09:12:00.000Z" }
```

Errors (422 fieldErrors): quantity, fromBucket, bucket, newCount, note · 409 would-be-negative stock.

9. Payments & invoices

32. GET

`/customers/{customerId}/payments/summary`

Used by: Both · **Auth:** Token (customer own / `payments.view`) · **Frontend:**
`paymentService.getCustomerPaymentSummary`

Response 200 — `CustomerPaymentSummary`.

```
{ "totalInvoiced": 14940, "totalPaid": 11400, "runningBalance": 3540, "overdueAmount": 0, "lastPaymentAt": "2026-06-11T14:25:00.000Z" }
```

33. GET /invoices

Used by: Both · **Auth:** Token (customer own / `payments.view`) · **Frontend:** `paymentService.listInvoices`

Query: `customerId`, `openOnly=true` (balance > 0). **Response 200** — `Invoice[]` newest `issuedAt` first.

```
[
  {
    "id": "INV-0144", "invoiceNumber": "MBGA/INV/0144", "orderId": "ORD-2606-0144",
    "orderNumber": "ORD-2606-0144",
    "customerId": "CUST-006", "customerName": "Annapurna Sweets & Namkeen",
    "amount": 7500, "gstPercent": 18, "gstAmount": 1350, "totalAmount": 8850,
    "paidAmount": 0, "balance": 8850, "status": "UNPAID",
    "issuedAt": "2026-06-13T18:00:00.000Z", "dueAt": "2026-06-20T18:00:00.000Z",
    "isGstInvoice": false
  }
]
```

34. GET /payments

Used by: Both · **Auth:** Token (customer own / `payments.view`) · **Frontend:** `paymentService.listPayments`

Query: `customerId`, `reconciliationStatus = PENDING | RECONCILED | INVALID | ALL`, `search`. **Response 200** — `Payment[]` newest `paidAt` first.

```
[
  {
    "id": "PAY-0211", "invoiceId": "INV-0141", "invoiceNumber": "MBGA/INV/0141",
    "customerId": "CUST-001", "customerName": "Sharma General Store",
    "invoiceAmount": 5400, "amountCollected": 5400, "mode": "UPI", "paidAt": "2026-06-11T14:25:00.000Z", "reference": "UPI-4471820039",
    "reconciliationStatus": "RECONCILED", "collectedBy": "Sunita Joshi",
    "customerRunningBalance": 3540
  }
]
```

35. GET /payments/{id}

Used by: Both · **Auth:** Token · **Frontend:** `paymentService.getPayment`

Response 200 — **Payment**. **Errors:** 404.

36. POST /payments

Used by: Merchant · **Auth:** `payments.record` (Manager, Accountant) · **Frontend:** `paymentService.recordPayment`

Request payload — **RecordPaymentRequest**

Field	Type	Required	Notes
invoiceId	string	✓	Must be open (balance > 0)
amount	Money	✓	> 0, ≤ invoice balance
mode	"CASH" "UPI" "NEFT"	✓	
reference	string	UPI/NEFT	Required for non-cash
paidAt	ISO date	✓	

```
{ "invoiceId": "INV-0144", "amount": 5000, "mode": "UPI", "reference": "UPI-9912003311", "paidAt": "2026-06-15T09:12:00.000Z" }
```

Response 201 — **Payment**.

```
{ "id": "PAY-0213", "invoiceId": "INV-0144", "invoiceNumber": "MBGA/INV/0144",  
  "customerId": "CUST-006", "customerName": "Annapurna Sweets & Namkeen",  
  "invoiceAmount": 8850, "amountCollected": 5000, "mode": "UPI", "paidAt": "2026-06-15T09:12:00.000Z", "reference": "UPI-9912003311",  
  "reconciliationStatus": "PENDING", "collectedBy": "Sunita Joshi",  
  "customerRunningBalance": 3850 }
```

Backend rules: updates invoice `paidAmount/balance/status`; CASH → **RECONCILED**, UPI/NEFT → **PENDING**; recomputes `customer.runningBalance`; notifies customer.

Errors: 422 `fieldErrors.amount` ("Enter a valid amount" / "Exceeds balance") · 422 `fieldErrors.reference` · 404 invoice.

10. Expenses — merchant

37. GET /expenses

Used by: Merchant · **Auth:** `expenses.view` (Manager, Accountant) · **Frontend:** `expenseService.listExpenses`

Query: `period = YYYY-MM | ALL, category = ExpenseCategory | ALL`. **Response 200** — `Expense[]` newest `date` first.

```
[
  { "id": "EXP-0312", "category": "FUEL", "amount": 4200, "date": "2026-06-15T08:00:00.000Z", "enteredBy": "Mohan Lal", "reportPeriod": "2026-06", "note": "Diesel - MP09 GH 4521" }
]
```

38. POST /expenses

Used by: Merchant · **Auth:** `expenses.add` · **Frontend:** `expenseService.addExpense`

Request payload — `CreateExpenseRequest`

Field	Type	Required	Notes
category	"FUEL" "VEHICLE_MAINTENANCE" "SALARY" "RENT" "UTILITIES" "MISC"	✓	
amount	Money	✓	> 0
date	ISO date	✓	App sends today
note	string		

```
{ "category": "FUEL", "amount": 1500, "date": "2026-06-15T09:12:00.000Z", "note": "Diesel" }
```

Response 201 — `Expense` (`reportPeriod = date.slice(0,7)`).

```
{ "id": "EXP-0313", "category": "FUEL", "amount": 1500, "date": "2026-06-15T09:12:00.000Z", "enteredBy": "Rajesh Verma", "reportPeriod": "2026-06", "note": "Diesel" }
```

Errors: 422 `fieldErrors.amount` · 422 `fieldErrors.category`.

11. Reports — merchant

39. GET /reports/{type}?from=&to=

Used by: Merchant · **Auth:** `reports.view` (Manager, Accountant) · **Frontend:** `reportService.getReport`

Path: `type` ∈ `DAILY_DELIVERY` | `PAYMENT_COLLECTION` | `PENDING_PICKUP` | `EXPENSE`. **Query:** `from`, `to` (ISO dates, inclusive).

Response 200 — Report (`type`, `title`, `period`, `summary[]`, `columns[]`, `rows[]` — each row must include `id`, `generatedAt`).

type	summary items	columns
DAILY_DELIVERY	Deliveries, Delivered, Cylinders, Empties collected	slip, customer, qty (right), status
PAYMENT_COLLECTION	Collected, Cash, UPI, NEFT (excl. INVALID)	date, customer, mode, amount (right), status
PENDING_PICKUP	Customers, Cylinders pending	customer, cylinder, pending (right), since
EXPENSE	Total expenses, Entries, Largest category	date, category, by, amount (right)

```
{
  "type": "PAYMENT_COLLECTION", "title": "Payment Collection Report",
  "period": { "from": "2026-06-01T00:00:00.000Z", "to": "2026-06-15T23:59:59.000Z" },
  "summary": [ { "label": "Collected", "value": "₹1,42,640" }, { "label": "Cash",
"value": "₹28,000" }, { "label": "UPI", "value": "₹79,640" }, { "label": "NEFT",
"value": "₹35,000" } ],
  "columns": [ { "key": "date", "label": "Date" }, { "key": "customer", "label":
"Customer" }, { "key": "mode", "label": "Mode" }, { "key": "amount", "label":
"Amount", "align": "right" }, { "key": "status", "label": "Status" } ],
  "rows": [ { "id": "PAY-0211", "date": "11 Jun 2026", "customer": "Sharma General
Store", "mode": "UPI", "amount": "₹5,400", "status": "Reconciled" } ],
  "generatedAt": "2026-06-15T09:12:00.000Z"
}
```

12. Notifications

The user is inferred from the token. Customers get their own bucket; staff share the merchant bucket.

40. GET /notifications

Used by: Both · **Auth:** Token · **Frontend:** `notificationService.list`

Response 200 — `AppNotification[]` newest first.

```
[
  { "id": "NTF-M-1", "title": "Stock below reorder threshold", "message": "47.5 KG filled stock is 42 against a threshold of 50. Plan a BPCL refill.", "severity": "CRITICAL", "category": "STOCK", "createdAt": "2026-06-15T07:32:00.000Z", "read": false },
  { "id": "NTF-M-2", "title": "Orders awaiting confirmation", "message": "1 new order (ORD-2606-0150, Hotel Blue Orchid) is waiting to be confirmed.", "severity": "WARNING", "category": "ORDER", "createdAt": "2026-06-15T08:00:00.000Z", "read": false, "referenceType": "ORDER", "referenceId": "ORD-2606-0150" }
]
```

41. GET /notifications/{id}

Used by: Both · **Auth:** Token · **Frontend:** `notificationService.get` **Response 200** — `AppNotification`. **Errors:** 404.

42. POST /notifications/{id}/read

Used by: Both · **Auth:** Token · **Frontend:** `notificationService.markRead` **Request:** empty. **Response:** 204.

43. POST /notifications/read-all

Used by: Both · **Auth:** Token · **Frontend:** `notificationService.markAllRead` **Request:** empty. **Response:** 204.

13. Merchant dashboard

44. GET /dashboard/merchant

Used by: Merchant · **Auth:** `dashboard.view` · **Frontend:** `dashboardService.getMerchantDashboard`

Response 200 — `MerchantDashboard`.

```
{
  "kpis": {
    "totalCustomers": 10, "pendingVerification": 2,
```

```

    "totalOrders": 14, "outForDelivery": 2, "todaysOrders": 3, "pendingOrders": 1,
    "readyForDispatch": 3, "deliveriesToday": 2,
    "pendingPaymentsAmount": 87640, "pendingPaymentsCount": 3, "pendingPickups": 6,
    "filledStock": 532, "emptyStock": 269, "damagedStock": 16, "pendingKyc": 2
  },
  "alerts": [ { "id": "NTF-M-1", "title": "Stock below reorder threshold", "severity":
"CRITICAL", "category": "STOCK", "message": "...", "createdAt": "...", "read": false } ],
  "recentOrders": [ { "id": "ORD-2606-0150", "...": "Order" } ],
  "recentRegistrations": [ { "id": "KYC-003", "...": "KycApplication" } ],
  "asOf": "2026-06-15T09:12:00.000Z"
}

```

KPI definitions: **pendingOrders** = status PLACED · **readyForDispatch** = CONFIRMED + PREPARING · **outForDelivery** = slips DISPATCHED · **deliveriesToday** = slips DISPATCHED/delivered today · **pendingPaymentsAmount/Count** = open invoices · **pendingPickups** = $\sum$ slip.pendingPickup · ***Stock** = $\sum$ across cylinder types · **pendingKyc** = PENDING applications · **alerts** = up to 4 unread non-INFO merchant notifications.

14. Document upload

45. POST /documents (multipart/form-data)

Used by: Both (customer registration, staff-added customer) · **Auth:** Token (customer NEW/PENDING or **customers.create**) · **Frontend:** replaces mock **attachDocument**

Form fields

Field	Type	Notes
file	binary	PDF/JPG/PNG, ≤ 5 MB
type	KycDocumentType (AADHAAR PAN FSSAI GST)	

Response 201

```

{ "fileId": "doc_01HZX...", "fileName": "aadhaar_front.jpg", "contentType":
"image/jpeg", "size": 348211 }

```

The app puts **fileId** into **documents[]**.**fileName** for endpoints 6 / 16.

Errors: 413 too large · 415 unsupported type.

46. GET /documents/{fileId}/url

Used by: Merchant (KYC review "view document") · **Auth:** Token (owner or `customers.kyc.review`)

Response 200

```
{ "url": "https://...signed...", "expiresInSeconds": 300 }
```

15. Common error envelope

Every non-2xx response body:

```
{
  "message": "Human-readable message shown to the user",
  "code": "OPTIONAL_MACHINE_CODE",
  "fieldErrors": { "fieldName": "Inline message shown under that input" }
}
```

HTTP status	Error kind	Typical use
400 / 422	VALIDATION	Bad input — use <code>fieldErrors</code> so the app can highlight/auto-focus the field
401	AUTHENTICATION	Missing/expired token → app signs the user out
403	AUTHORIZATION	Role/permission denied, customer not approved, <code>WRONG_APP</code> , <code>USER_INACTIVE</code>
404	NOT_FOUND	Unknown id
409	CONFLICT	State conflict (already reviewed, already delivered, duplicate mobile, insufficient stock)
5xx	SERVER	Generic failure → toast "Something went wrong on our side"

Machine codes the frontend reacts to specially: `WRONG_APP`, `USER_INACTIVE`, `OTP_INVALID`, `OTP_EXPIRED`.

Derived from `docs/API/API_SPEC.md` (source of truth for enums, full data-model field tables, and business rules). Keep both files in sync when backend paths/shapes change.